

COER UNIVERSITY, ROORKEE

Department of Computer Science & Engineering

Computer Vision Laboratory

Labsheet . 1

Implementation of Fundamental Image Processing Operations using Python and OpenCV

Submitted By: Utsav Kumar Kushwaha

Course: B.Tech CSE

Academic Session: 2026-27

Experiment No. 1

Aim

To develop a comprehensive understanding of fundamental image processing by implementing image acquisition, storage, color-space conversion, geometric transformations, image complement generation, and Region of Interest extraction using Python and OpenCV.

Purpose

Digital image processing is the foundation of modern computer vision systems. Every intelligent vision application—from autonomous vehicles and medical diagnosis to face recognition and industrial inspection—begins with preprocessing. This experiment helps students understand how images are represented as pixels, how OpenCV performs common transformations, and why preprocessing improves the accuracy and efficiency of later computer vision tasks.

Learning Outcomes

- Load and display images using OpenCV and Matplotlib.
- Understand image properties such as resolution and channels.
- Convert images into multiple color spaces.
- Perform resizing, rotation, and flipping operations.
- Generate negative images and extract ROI.
- Compare processed images and interpret their significance.

Theory

A digital image is a two-dimensional matrix of pixels. Each pixel stores intensity information. A grayscale image contains one intensity value per pixel, while a color image stores three values corresponding to color channels. OpenCV stores color images in BGR format instead of RGB. Before displaying an image with Matplotlib, the channels are converted from BGR to RGB.

Image preprocessing improves visual quality, removes unnecessary information, reduces computational cost, and makes feature extraction more reliable. Common preprocessing operations include resizing, grayscale conversion, contrast enhancement, filtering, normalization, and color-space transformation.

Prerequisites

- Basic Python programming
- Knowledge of pixels and color channels
- OpenCV (cv2), NumPy, and Matplotlib
- VS Code, Jupyter Notebook, or Google Colab

Software Required

Software	Version	Purpose
Python	3.x	Programming
OpenCV	4.x	Image Processing
NumPy	Latest	Matrix Operations
Matplotlib	Latest	Visualization
VS Code/Jupyter	Latest	Development

Algorithm

1. Import required libraries.
2. Load the input image.
3. Display the image.
4. Check image properties.
5. Save image in JPEG and PNG.
6. Convert image into Grayscale, HSV, and LAB.
7. Perform resize, rotation, and flip.
8. Generate the negative image.
9. Crop a Region of Interest.
10. Display all outputs together and analyze the results.

Python Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("image.jpg")

plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.show()

print("Shape:", image.shape)
print("Data Type:", image.dtype)

cv2.imwrite("output.jpg", image)
cv2.imwrite("output.png", image)

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

resized = cv2.resize(image, (400,300))

(h,w)=image.shape[:2]
matrix=cv2.getRotationMatrix2D((w//2,h//2),45,1)
rotated=cv2.warpAffine(image,matrix,(w,h))

horizontal=cv2.flip(image,1)
vertical=cv2.flip(image,0)

negative = 255-image

roi=image[100:300,200:400]
```

```
plt.figure(figsize=(12,8))
images=[cv2.cvtColor(image,cv2.COLOR_BGR2RGB),gray,
        cv2.cvtColor(hsv,cv2.COLOR_HSV2RGB),
        cv2.cvtColor(lab,cv2.COLOR_LAB2RGB),
        cv2.cvtColor(negative,cv2.COLOR_BGR2RGB),
        cv2.cvtColor(horizontal,cv2.COLOR_BGR2RGB)]
titles=["Original","Gray","HSV","LAB","Negative","Flip"]

for i,img in enumerate(images):
    plt.subplot(2,3,i+1)
    if i==1:
        plt.imshow(img,cmap="gray")
    else:
        plt.imshow(img)
    plt.title(titles[i])
    plt.axis("off")
plt.show()
```

➤ Step-by-Step Procedure with Screenshot Placeholders

Step 1: Import Libraries

Import OpenCV, NumPy, and Matplotlib successfully.

Step 2: Load Image

Load the image from the local system using cv2.imread().

Step 3: Display Original

Display using OpenCV and Matplotlib.

Step 4: Image Properties

Check shape, channels, and data type.

Step 5: Save Image

Save in JPG and PNG formats.

Step 6: Grayscale Conversion

Convert to grayscale.

Step 7: HSV Conversion

Convert to HSV color space.

Step 8: LAB Conversion

Convert to LAB color space.

Step 9: Resize

Resize the image.

Step 10: Rotation

Rotate the image by 45 degrees.

Step 11: Horizontal & Vertical Flip

Perform both flip operations.

Step 12: Negative Image

Generate complement image.

Step 13: ROI Extraction

Crop a Region of Interest.

Step 14: Comparison

Compare all processed images.

Step 1: Import Libraries

```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt

In [2]: print("Libraries imported successfully!")
```

Libraries imported successfully!

Step 2: Load Image

```
In [3]: img = cv2.imread('sample.jpg')
print("Image loaded successfully!")
```

Image loaded successfully!

```
In [4]: img is not None
```

Out[4]: True

Step 3: Display Original

OpenCV Display



Matplotlib Display



Step 4: Image Properties

```
In [5]: print("Shape:", img.shape)
print("Channels:", img.shape[2])
print("Data Type:", img.dtype)
```

Shape: (720, 1280, 3)
Channels: 3
Data Type: uint8

Step 5: Save Image

```
In [6]: cv2.imwrite('output.jpg', img)
cv2.imwrite('output.png', img)
print("Images saved in JPG and PNG formats!")
```

Images saved in JPG and PNG formats!

File	Size	Format
output.jpg	187 KB	JPG File
output.png	758 KB	PNG File

Step 6: Grayscale Conversion

```
In [7]: gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.imshow(gray, cmap='gray')
plt.title('Grayscale Image')
plt.axis('off')
plt.show()
```



Step 7: HSV Conversion

```
In [8]: hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
plt.imshow(hsv)
plt.title('HSV Image')
plt.axis('off')
plt.show()
```



Step 8: LAB Conversion

```
In [9]: lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
plt.imshow(lab)
plt.title('LAB Image')
plt.axis('off')
plt.show()
```



Step 9: Resize

```
In [10]: resized = cv2.resize(img, (640, 360))
plt.imshow(resized, cv2.COLOR_BGR2RGB)
plt.title('Resized Image (640x360)')
plt.axis('off')
plt.show()
```



Step 10: Rotation

```
In [11]: (h, w) = img.shape[:2]
center = (w//2, h//2)
M = cv2.getRotationMatrix2D(center, 45, 1.0)
rotated = cv2.warpAffine(img, M, (w, h))
plt.imshow(rotated, cv2.COLOR_BGR2RGB)
plt.title('Rotated Image (45°)')
plt.axis('off')
plt.show()
```



Step 11: Horizontal & Vertical Flip

```
In [12]: flip_h = cv2.flip(img, 1)
flip_v = cv2.flip(img, 0)
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(flip_h, cv2.COLOR_BGR2RGB))
plt.title('Horizontal Flip')
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(flip_v, cv2.COLOR_BGR2RGB))
plt.title('Vertical Flip')
plt.axis('off')
plt.show()
```



Step 12: Negative Image

```
In [13]: negative = 255 - img
plt.imshow(negative, cv2.COLOR_BGR2RGB)
plt.title('Negative Image')
plt.axis('off')
plt.show()
```



Step 13: ROI Extraction

```
In [14]: x, y, w, h = 200, 150, 400, 300
roi = img[y:y+h, x:x+w]
plt.imshow(roi, cv2.COLOR_BGR2RGB)
plt.title('ROI (Cropped Region)')
plt.axis('off')
plt.show()
```



Step 14: Comparison of All Processed Images

Original	Grayscale	HSV	LAB	Resized (640x360)	Rotated (45°)	Horizontal Flip	Vertical Flip	Negative	ROI (Cropped)
									

Observations

Operation	Observation	Significance
Original	Image loaded successfully	Starting point
Grayscale	Color removed	Reduces computation
HSV	Hue separated	Useful for color detection
LAB	Perceptual color	Improves segmentation
Resize	Dimensions changed	Standardizes input
Rotate	Orientation changed	Data augmentation
Flip	Mirror image	Training diversity
Negative	Pixel inversion	Medical/document analysis
ROI	Selected region extracted	Focused processing

Advantages of Image Preprocessing

- Improves image quality.
- Reduces noise.
- Enhances important features.
- Reduces computational complexity.
- Improves machine learning accuracy.
- Standardizes images before training.
- Supports real-time computer vision.

Limitations

- Excessive resizing may reduce details.
- Improper color conversion may lose information.
- Cropping may remove useful objects.
- Repeated compression affects image quality.

Applications

- Face Recognition
- Medical Imaging
- Autonomous Vehicles
- Traffic Monitoring
- Industrial Quality Inspection
- Satellite Image Analysis
- Document OCR
- Barcode Scanning
- Agriculture Monitoring
- Robotics Vision

Precautions

- Use a high-quality input image.
- Convert BGR to RGB before Matplotlib display.
- Maintain aspect ratio while resizing.
- Verify ROI coordinates.
- Save outputs before closing the program.

Result

The experiment was successfully performed using Python and OpenCV. Fundamental preprocessing operations including image loading, color-space conversion, resizing, rotation, flipping, image complement generation, and ROI extraction were implemented successfully. The experiment demonstrated that preprocessing improves image quality and prepares data for advanced computer vision tasks.

➤ Viva Questions with Detailed Answers

1. What is a digital image? Differentiate between grayscale and color images.

A digital image is a collection of pixels arranged in rows and columns. A grayscale image has one intensity channel, whereas a color image has three channels representing color information.

2. Explain RGB, BGR, HSV, and LAB.

RGB is used for displays, BGR is OpenCV's default order, HSV separates color from brightness, and LAB represents colors closer to human perception.

3. Why convert an image to grayscale?

Grayscale removes unnecessary color information and reduces computation while preserving structural information.

4. What is image complement?

Image complement creates a negative image by replacing each pixel with 255 minus its value. It is useful in medical imaging and document enhancement.

5. Difference between resizing, cropping, and scaling.

Resizing changes image dimensions, cropping removes a selected portion, and scaling enlarges or shrinks while maintaining proportions.

6. What is ROI?

ROI is a selected part of an image used for focused analysis, reducing processing time.

7. Why is OpenCV preferred?

OpenCV is fast, optimized in C++, supports real-time processing, and provides hundreds of computer vision functions.

8. How do resolution and pixel intensity affect quality?

Resolution determines detail, while pixel intensity controls brightness and contrast.

9. Mention five real-world applications.

Face recognition, OCR, medical imaging, surveillance, and autonomous driving.

10. How does preprocessing improve deep learning?

It reduces noise, standardizes inputs, highlights useful features, and improves model accuracy and training speed.